API INTEGRATION & REFERENCE REPORT

LED Auction Results API

Reverse-engineered integration documentation for the Legal Execution Department auction results service at asset.led.go.th/report/reports.asp: verified pages, endpoint reference, field-level data dictionary, code samples, OpenAPI facade, and the legal guardrails for responsible programmatic access.

Prepared for Developer Integration Teams

Bilingual edition: English · ภาษาไทย

OCTOBER 2026 · VERSION 1.0

Table of Contents

1. Executive Summary / บทสรุปผู้บริหาร 1
1.1 Key findings 1
1.2 What you receive with this report 1
1.3 Executive summary in Thai / บทสรุปผู้บริหาร 2
2. System Overview and Context 3
2.1 Where this service sits in the LED landscape 3
2.2 Technology profile 4
2.3 The data domain 4
3. Architecture and Request Flow 5
3.1 Lifecycle walkthrough 5
3.2 Character encoding 6
3.3 The DevTools verification loop 6
4. Endpoint Reference 7
4.1 Endpoint inventory 7
4.2 E1 - reports.asp (per-case search form) 7
4.3 E2 - the results handler (inferred) 8
4.4 E3 / E4 - office-and-date browsers 8
4.5 E5 / E6 - companion catalogue endpoints 9
5. Field-Level Data Dictionary 9
5.1 Search parameters 9
5.2 Anatomy of a red case number 10
5.3 Result record fields (expected shape) 10
5.4 Enforcement office reference 11
6. Code Samples 11
6.1 cURL - quick verification 11
6.2 Python - production scaffold 12
6.3 JavaScript - Node 18 fetch 14
6.4 Politeness pattern 15
7. OpenAPI Specification Guide 16
7.1 How the file is organised 16
7.2 Working with the spec 16

8. Legal, Ethics and Responsible Use 17

8.1 The site's own notice 17

8.2 Personal data under the PDPA 17

8.3 Responsible-use rules of thumb 17

9. Risks, Limitations and Mitigations 18

9.1 Reading the top risks 19

10. Appendix 19

A. Enforcement office list (sample) 19

B. Verified case-number examples 20

C. Go-live checklist 20

D. Reference URLs 21

E. Bundle file map 21

CHAPTER 1 - OVERVIEW AND KEY FINDINGS

1. Executive Summary / บทสรุปผู้บริหาร

This report documents the programmatic interface of the auction results service published by the Legal Execution Department (กรมบังคับคดี, LED) of the Thai Ministry of Justice at `asset.led.go.th/report/reports.asp`. The page is the department's public “search auction sale results per case” facility (ค้นหาผลการขายทอดตลาดรายคดี), and this document treats it as the integration target for developers who need to retrieve auction outcome data in a structured, repeatable way. Because the department does not publish an official REST or JSON API for this service, the report reconstructs the interface from the live system: it maps the search forms, the visible field labels, the server-side page family under `/report/`, and the operational and legal constraints that any integration must respect.

1.1 Key findings

Five findings shape everything that follows. They are based on direct inspection of the live pages, on official LED communications that quote these URLs, and on the behaviour observed when the system is accessed from outside the government network.

- **No official public API exists for this service.** The system is a classic ASP web application that returns server-rendered HTML. Any integration is therefore a form-emulation and HTML-extraction client, not a JSON consumer.
- **The /report/ family has three user-facing pages.** `reports.asp` searches results per case (red case number + Buddhist year, optionally plaintiff and sale date); `report.asp` and the newer `report_new/report.asp` browse results by enforcement office and sale date.
- **Search parameters must be verified once in a browser.** The exact form-field names and the submission endpoint could not be captured programmatically during this study (the origin throttles non-browser clients), so parameter names in this report are marked as inferred and Chapter 3 gives a five-minute DevTools procedure to pin them down.
- **The site carries a restrictive copyright notice.** LED claims the data as its property and prohibits unauthorised use, reproduction, or modification. Personal names of case parties also fall under the Thai Personal Data Protection Act. Chapter 8 turns this into practical rules.
- **Availability is best-effort.** The origin responded slowly or timed out repeatedly during the study when accessed through datacentre networks. Integrations need generous timeouts, retries with backoff, and a local cache.

1.2 What you receive with this report

The report ships as a bundle so that the documentation and the working artifacts never drift apart. The PDF you are reading is the reference document; the companion files are machine-readable and

code-level assets that implement the recommendations described in Chapters 6 and 7. Keep the bundle together and update the OpenAPI file whenever the live form is re-verified, so that the contract stays truthful.

Artifact	File	Purpose
Integration report (this PDF)	LED_Auction_Results_API_Report.pdf	Full reference: endpoints, data dictionary, code, legal guidance
OpenAPI 3.0 specification	openapi/led-auction-openapi.yaml	Machine-readable facade of the legacy endpoints and parsed records
Python client	samples/led_auction_client.py	Session, retry, encoding, rate-limit and parse scaffold
cURL samples	samples/led_auction_curl.sh	Copy-paste shell requests for quick verification
JavaScript sample	samples/led_auction_fetch.mjs	Node 18+ fetch example with HTML post-processing notes
Cover source	assets/cover.html	Editable HTML source of the report cover page

Table 1.1 - Deliverables in this documentation bundle.

3	0	2	1
verified pages in the /report/ family	official JSON/REST endpoints	companion code clients	OpenAPI 3.0 facade spec

1.3 Executive summary in Thai / บทสรุปผู้บริหาร

รายงานนี้อธิบายการเชื่อมต่อระบบรายงานผลการขายทอดตลาดของกรมบังคับคดี ที่เผยแพร่บนเว็บไซต์ asset.led.go.th หน้า reports.asp ซึ่งเป็นระบบที่พัฒนาด้วย ASP แบบเก่า และไม่มี API รูปแบบเปิดอย่างเป็นทางการสำหรับภายนอก
ผู้ที่ต้องการดึงข้อมูลจึงต้องส่งฟอร์มค้นหาและแยกข้อมูลจากหน้า HTML ที่ตอบกลับมา

ข้อมูลสำคัญที่ทีมพัฒนาต้องรู้คือ ชื่อพารามิเตอร์ของฟอร์มยังไม่ได้รับการยืนยันจากระบบโดยตรง จึงต้องตรวจสอบด้วย DevTools ก่อนใช้งานจริง ตามขั้นตอนในบทที่ 3 นอกจากนี้
เว็บไซต์ระบุว่าข้อมูลทุกอย่างเป็นกรรมสิทธิ์ของกรมบังคับคดี ห้ามนำไปใช้ ทำซ้ำ หรือดัดแปลงย่อยียนไม่ได้รับอนุญาต ดังนั้นควรติดต่อขออนุญาตหรือใช้ช่องทางข้อมูลภาครัฐ (data.go.th) สำหรับการใช้งานระดับใหญ่

CHAPTER 2 - THE PLATFORM BEHIND THE ENDPOINT

2. System Overview and Context

The Legal Execution Department is the agency responsible for enforcing court judgments in Thailand: seizing assets, managing bankruptcy estates, and selling seized property at public auction. Its electronic services (บริการประชาชน e-Service) are reached through the departmental portal at led.go.th, while the auction-related systems live on the dedicated host asset.led.go.th. The host root redirects to `/newbidreg/`, the searchable catalogue of assets advertised for auction, and the `/report/` path studied here serves the post-auction counterpart: the official record of what happened at each sale.

2.1 Where this service sits in the LED landscape

LED operates a family of public systems that share the same visual framework and the same case-number domain. The e-Service catalogue on the main portal lists the case-status inquiry, the receiver announcements, e-Filing, queue booking, and several auction-related services. Within that family, the report system answers the question “what happened to the asset offered at auction on date X in case Y?”. Practitioners - asset-management companies, lawyers, journalists, and bidders - reach it from auction announcements that quote its URLs directly, which is how the two report pages acquired their stable, widely-circulated addresses.

Page (path)	Thai title	Function	Search keys
/report/reports.asp	ค้นหาผลการขายทอดตลาดรายคดี	Per-case search of auction sale results	Red case number + Buddhist year; optional plaintiff and sale date
/report/report.asp	รายงานผลการขายทอดตลาด	Browse results by office and sale date	Enforcement office (dropdown) + sale date
/report_new/report.asp	รายงานผลการขายทอดตลาด (new)	Newer variant of the office-and-date browser	Enforcement office (dropdown) + sale date

Table 2.1 - The three verified pages of the /report/ family.

All three pages publish the identical footer declaration that data on the site is the property of the department and may not be used, reproduced, or adapted without permission. This declaration matters for integration planning and is examined in Chapter 8 together with the Thai legal instruments that surround it. The pages also share the same master layout, which indicates one underlying ASP application rather than unrelated systems.

2.2 Technology profile

Several observable characteristics define the integration surface. First, the .asp extensions and the form-driven flow identify a classic ASP application on Microsoft IIS: session state is cookie-based, pages are fully server-rendered, and there is no JavaScript single-page framework to reverse-engineer. Second, the system is reachable only over HTTPS on the standard port; no developer or documentation path is advertised on the host. Third, the origin is slow from some networks - during this study, direct connections from a datacentre repeatedly timed out while the pages remained reachable through normal browsing paths - which suggests per-connection throttling or geographic filtering rather than an intentionally rate-limited public API.

- Transport: HTTPS only, standard port 443; no client certificate required; server certificate issued for asset.led.go.th.
- Rendering: server-side HTML with Thai labels; no visible client-side framework; forms submit to .asp handlers.
- State: classic ASP session cookie issued on first request; keep a cookie jar across the form and the submission call.
- Observed reliability: multi-second latencies and intermittent connection timeouts from datacentre IPs; plan for 30-60 second timeouts and exponential backoff.
- Known protective control: the sibling search system /newbidreg/ prompts for a confirmation code (รหัสยืนยัน, a captcha) on its search form; the report pages studied here expose no captcha, but the control may be extended to them at any time.

2.3 The data domain

The domain of the report system is the outcome of judicial auction sales. Each sale is anchored to a red case number (หมายเลขคดีแดง), the identifier under which a court enforcement file is publicly tracked; the records reference the enforcement office that conducted the sale, the scheduled sale date, and the result of each bidding round. Around the core result, the ecosystem’s pages also expose asset descriptors - property type, location, title-document number, appraised price - as seen in the asset-detail pages of the companion catalogue. A well-shaped integration should therefore model two entities: the search request (case, office, date window) and the result record (case identity, sale instance, outcome and prices), which is exactly the split used in the OpenAPI facade of Chapter 7.

CHAPTER 3 - HOW AN INTEGRATION ACTUALLY TALKS TO THE SYSTEM

3. Architecture and Request Flow

Because the service is a form-driven ASP application, every integration follows the same six-step lifecycle: fetch the form, submit search parameters, receive an HTML results page, extract the rows, normalise them, and store them as structured records. Figure 3.1 shows the flow together with the one human step that makes the whole design safe: a one-time verification of the exact parameter names in the browser's network inspector before any code is written against the live endpoint.

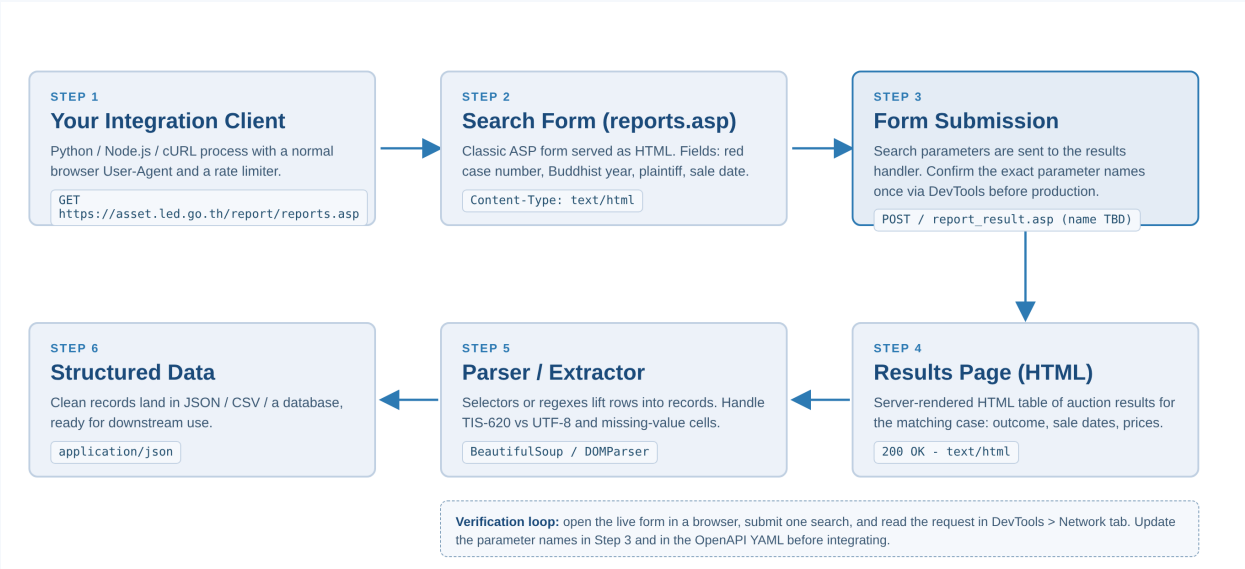


Figure 3.1 - Integration request flow against the reports.asp form, with the DevTools verification loop.

3.1 Lifecycle walkthrough

Step 1 issues a plain GET to the form page and keeps the session cookie. Step 2 parses the returned HTML only enough to confirm the field names - a stable integration does not scrape the form itself at runtime. Step 3 submits the parameters with the same content type the browser uses (form-urlencoded by default in classic ASP). Step 4 receives the results page; HTTP 200 with a zero-result table is a valid outcome and must not be treated as an error. Step 5 extracts rows with selectors anchored to stable Thai headers, and Step 6 normalises values: Buddhist years to ISO dates, baht amounts to integers, and office names to their canonical codes.

Two transport details deserve emphasis. The cookie jar matters because classic ASP sessions validate that the submission came from the same session that loaded the form; dropping cookies between the two calls is the most common cause of empty results in such integrations. And because the origin is slow under load, every network call should carry an explicit timeout and a retry policy with exponential backoff and jitter - the Python client in Chapter 6 implements both.

3.2 Character encoding

Thai government ASP systems historically served windows-874 (TIS-620) responses, while the LED portal family is served as UTF-8. The safe rule is a three-stage detection: trust the HTTP Content-Type charset when present; otherwise look for the meta declaration in the first 2 KiB of the body; otherwise sniff with a detector and default to UTF-8. Whatever the outcome, decode explicitly and immediately - never let the HTTP client guess at decode time. Mismatched decoding does not raise errors; it silently corrupts Thai names (โจทก becomes mojibake), which then poisons downstream matching. The Python sample implements exactly this ladder.

3.3 The DevTools verification loop

The report pages could not be captured in raw HTML form during this study because the origin throttles non-browser clients, so the submission endpoint and parameter names are marked as inferred throughout. Closing that gap takes about five minutes with a browser, and the procedure should be repeated whenever the department deploys changes:

- Open `https://asset.led.go.th/report/reports.asp` in Chrome or Firefox and press F12 to open DevTools.
- Switch to the Network tab, enable “Preserve log”, and filter by the .asp extension.
- Fill the form with a real case you know exists - red case number and Buddhist year are mandatory per the page’s own validation message - and press ค้นหา (Search).
- Select the top request that appears (it will be the results page), read its method, URL, and Form Data / Query String Parameters panel, then copy the exact field names and values.
- Reproduce the call once with cURL using the captured names; when the response matches the browser view, record the parameters in the OpenAPI YAML and in your client constants.

verification-notes.txt

```
# After verification, pin the captured contract in your code:
# submission URL : (from DevTools, e.g. /report/report_result.asp)
# method : POST or GET (as captured)
# fields : case_no=..., year_be=..., plaintiff=..., sale_date=...
# Mark every name with a comment 'verified YYYY-MM-DD'.
```

CHAPTER 4 - VERIFIED PAGES, INFERRED CONTRACTS

4. Endpoint Reference

This chapter is the operational core of the report. It lists every endpoint relevant to the auction-results scope, distinguishes rigorously between what was verified (the page exists and its visible form labels were captured) and what is inferred (the submission handler and parameter names, pending the DevTools verification of Section 3.3), and gives the validation rules that the application itself announces on screen.

4.1 Endpoint inventory

#	Method	Path	Purpose	Status
E1	GET	`/report/reports.asp`	Per-case results search form	Verified
E2	GET/POST	`/report/ (results handler)`	Executes the per-case search; name to be captured	Inferred
E3	GET	`/report/report.asp`	Office-and-date results browser form	Verified
E4	GET/POST	`/report_new/report.asp (family)`	Newer variant of the office-and-date browser	Verified
E5	GET	`/newbidreg/default.asp`	Auction asset catalogue (context; out of scope)	Verified
E6	GET	`/newbidreg/asset_open.asp`	Asset detail page linked from announcements	Verified

Table 4.1 - Endpoint inventory. “Verified” = page reached and rendered during this study; “Inferred” = contract reconstructed from classic ASP patterns.

4.2 E1 - reports.asp (per-case search form)

The form presents four labelled inputs. The page states its own validation rules in two messages: “กรุณาใส่หมายเลขคดีแดง และ ปี พ.ศ. ที่ท่านต้องการค้นหาให้ครบถ้วน” (please enter both the red case number and the Buddhist year completely) and “หากใส่ชื่อโจทกกรุณาเลือกวันที่ขายหรือใส่เลขคดีแดงด้วย” (if entering a plaintiff name, also choose the sale date or the case number). These two sentences define the mandatory-field matrix below and should be mirrored in client-side validation to avoid wasted requests.

Label (Thai)	Meaning	Input type	Required	Notes
หมายเลขคดีแดง	Red case number	text	Yes*	Format prefix/number/B.E. year; see 5.2

Label (Thai)	Meaning	Input type	Required	Notes
ปี พ.ศ.	Year (Buddhist Era)	select (“-- เลือกปี พ.ศ. --”)	Yes*	Dropdown; pick the year of the sale
โจทก	Plaintiff	text	Conditional	Needs sale date or case number alongside
วันที่ขาย	Sale date	date/text	Conditional	Required when plaintiff is supplied

Table 4.2 - reports.asp form fields. * Required jointly: the page demands case number AND year together.

4.3 E2 - the results handler (inferred)

Classic ASP applications of this generation either post back to the same file or forward to a sibling such as a result or detail page. Both patterns are visible across the Thai government web estate, so this report deliberately does not guess a single name. The contract to capture in DevTools consists of five facts: the target URL, the HTTP method, the field names, the value formats (for example whether the year is sent as “2568” or “2545”), and any hidden fields the form injects. Until those are captured, treat every parameter name in this document as provisional and keep them in one constants block, exactly as the companion Python client does.

4.4 E3 / E4 - office-and-date browsers

report.asp and its newer variant report_new/report.asp publish the second search mode: choose an enforcement office (สำนักงานบังคับคดี/กอง) from a dropdown that enumerates the department’s offices and branches, pick a sale date, and press ดูผลการขาย (view sale results). The dropdown lists well over a hundred entries - provincial offices, Bangkok civil-execution divisions, bankruptcy divisions 1-6, rehabilitation office, and provincial branches - and the same list appears in the newer template. The page’s helper message “กรุณากรอกเงื่อนไขเพื่อค้นหา” asks the user to complete the conditions before searching. A snapshot of the office list is given in the Appendix and as an enum in the OpenAPI file; for new work, prefer E4 after verification, since the LED provincial offices already point their announcements at the /report_new/ path.

4.5 E5 / E6 - companion catalogue endpoints

The asset catalogue at /newbidreg/ is out of scope by the user's selection, but two of its addresses belong in any reference table because announcements and asset-management sites cite them as the authoritative detail link for each property. default.asp is the searchable list (asset list, map search, red-case search, today's sales, and the selling-office filter), and asset_open.asp renders the detail page for one advertised asset - the page whose data field set (appraisal prices, deposit, sale rounds) complements the results records of this report. Note that the catalogue's search form prompts for a confirmation code, which makes bulk automated use of E5 inappropriate; the report family has no such prompt today.

CHAPTER 5 - REQUEST PARAMETERS AND RECORD FIELDS

5. Field-Level Data Dictionary

This chapter fixes the vocabulary of the integration: how the search parameters are formed, how a red case number is built, which fields a result record should carry once parsed, and how the enforcement offices are identified. The dictionary is the single place where naming is defined; the OpenAPI file, the Python client, and the parser all reuse these names verbatim so that a change happens in exactly one commit.

5.1 Search parameters

Proposed name	Source label	Type	Format / domain	Example
`case_no`	หมายเลขคดีแดง	string	prefix/number/year, see 5.2	ย.1151/2545
`year_be`	ปี พ.ศ.	integer	Buddhist Era year, 4 digits	2568
`plaintiff`	โจทก	string	free text, partial allowed	บริษัท ...
`sale_date`	วันที่ขาย	date	site display format dd/mm/yyyy (verify)	11/08/2569
`office`	สำนักงานบังคับคดี/กอง	string/code	dropdown entry; keep Thai label + own code	ภาเก็ต / KRABI

Table 5.1 - Canonical search parameters. Names are proposals for your schema; map them to the captured form names after verification.

5.2 Anatomy of a red case number

The red case number is the public key of enforcement in Thailand. It concatenates a court-and-case-type prefix, a running number, and the Buddhist-Era year of registration, separated by a slash. Verified examples from public announcements include ย.1151/2545 (prefix ย., running number 1151, B.E. 2545) and ผบ.1730/2562 (prefix ผบ., number 1730, B.E. 2562). Prefixes identify the issuing court or execution unit and vary across the kingdom, so clients should store the string intact, match case-insensitively after trimming whitespace, and normalise only the Thai abbreviations they explicitly know. Because the year component is BE, any conversion to Gregorian years subtracts 543.

Component	Example	Description
Prefix	ย. / ผบ. / ...	Court or execution case-type abbreviation (Thai)
Running number	`1151 / 1730`	Sequential file number within the year
Year (B.E.)	`2545 / 2562`	Buddhist Era registration year; CE = BE - 543

Table 5.2 - Red case number structure with verified examples.

5.3 Result record fields (expected shape)

The results page is produced per case or per office-date window, and its rows describe the fate of each sale. The field set below is the expected shape compiled from the report pages' purpose, the department's announcement practice, and the asset-detail pages of the catalogue; capture the exact columns with the DevTools loop and adjust the parser once. Modelling these fields up front means the verification session only has to rename, not redesign.

Field	Type	Description	Caveats
`case_no`	string	Red case number of the enforcement file	Keep original Thai prefix
`office`	string	Conducting enforcement office / division	Dropdown label; keep Thai
`sale_date`	date	Scheduled and actual sale date of the round	B.E. display; normalise
`sale_round`	integer	Bidding round (1st, 2nd, ...)	Rounds reprice per rules
`asset_desc`	string	Short asset description (type, location)	Free text; trim
`start_price`	decimal	Opening price for the round (THB)	Strip commas before parse
`appraised_price`	decimal	Official appraised value (THB)	May be absent on some rows
`outcome`	string	Sold / not sold / withdrawn / postponed	Thai wording; map to enum

Field	Type	Description	Caveats
`bidder`	string	Winning bidder when sold	Personal data - see Chapter 8
`final_price`	decimal	Hammer price when sold	THB

Table 5.3 - Expected result-record fields; confirm column names during verification before freezing the parser.

5.4 Enforcement office reference

The office dropdown is the only enumeration the service exposes. It is stable, published on both E3 and E4, and doubles as a natural partition key for storage and for scheduling polite crawls (one office per window). The full observed list - provincial offices, their branches, the Bangkok civil-execution divisions, bankruptcy divisions, the debtor-rehabilitation office, and central placement units - is reproduced in Appendix A and embedded as a YAML enum in the OpenAPI file. Store both the Thai label and your own short code; the service itself does not publish machine codes, so the code space is yours to define.

CHAPTER 6 - CURL, PYTHON AND JAVASCRIPT CLIENTS

6. Code Samples

The samples below implement the lifecycle of Chapter 3 with the operational guardrails of this report baked in: session cookies, explicit timeouts, retry with exponential backoff, encoding detection, and rate limiting. Parameter names appear exactly once per file in a constants block marked as pending verification, so the DevTools session of Section 3.3 becomes a one-line change. The full Python client ships as a companion file and is quoted here in its essential parts.

6.1 cURL - quick verification

Use cURL to confirm the captured contract before writing any code. The first call loads the form and stores the session cookie; the second reproduces the browser submission. Replace the placeholder field names with the ones captured in DevTools, and keep the human User-Agent: the origin throttles non-browser clients.

curl - verify the reports.asp contract

```
# Step 1 - fetch the form and keep the session cookie
curl -sS -c cookies.txt \
-A 'Mozilla/5.0 (Windows NT 10.0; Win64; x64)' \
'https://asset.led.go.th/report/reports.asp' -o form.html

# Step 2 - submit the search (names TBD via DevTools, Section 3.3)
curl -sS -b cookies.txt \
-A 'Mozilla/5.0 (Windows NT 10.0; Win64; x64)' \
--data-urlencode 'case_no=1151/2545' \
--data-urlencode 'year_be=2545' \
'https://asset.led.go.th/report/.asp' \
-o result.html -w 'HTTP %{http_code} in %{time_total}s\n'

# If the response is TIS-620/widows-874, transcode before diffing
iconv -f TIS-620 -t UTF-8 result.html -o result.utf8.html
```

Two practical notes. First, when the submission turns out to be a GET, move the fields into the query string and drop `--data-urlencode`; the rest of the recipe is unchanged. Second, diff `result.html` against the browser's rendered output once; agreement at the row level is the definition of a verified contract.

6.2 Python - production scaffold

The companion client wraps requests.Session with the guardrails and keeps every site-specific fact in one constants block. The extract hook is deliberately left as a stub: HTML structure is the one thing that cannot be fixed until the results page is captured, and isolating it means the verification session touches a single function.

led_auction_client.py - part 1 of 4: constants

```
import time, random, logging
import requests
from bs4 import BeautifulSoup

BASE = 'https://asset.led.go.th'
FORM_URL = BASE + '/report/reports.asp'
# TODO(verify): capture in DevTools, Section 3.3 of the report
RESULT_URL = BASE + '/report/report_result.asp' # placeholder
FIELD_NAMES = {'case_no': 'case_no', # placeholder
'year_be': 'year_be', # placeholder
'plaintiff': 'plaintiff', # placeholder
'sale_date': 'sale_date'} # placeholder
UA = ('Mozilla/5.0 (Windows NT 10.0; Win64; x64) '
'AppleWebKit/537.36 (KHTML, like Gecko) Chrome/124.0 Safari/537.36')
```

led_auction_client.py - part 2 of 4: session and throttle

```
class LEDAuctionClient:
def __init__(self, min_interval=2.5, timeout=45, retries=4):
self.http = requests.Session()
self.http.headers.update({'User-Agent': UA,
'Accept-Language': 'th,en;q=0.8'})
self.min_interval = min_interval
self.timeout = timeout
self.retries = retries
self._last_call = 0.0

def _throttle(self):
wait = self.min_interval - (time.monotonic() - self._last_call)
if wait > 0:
time.sleep(wait + random.uniform(0, 0.75)) # jitter
self._last_call = time.monotonic()
```

led_auction_client.py - part 3 of 4: retry and charset ladder

```
def _get(self, url, **kw):
for attempt in range(self.retries):
self._throttle()
try:
r = self.http.get(url, timeout=self.timeout, **kw)
r.raise_for_status()
return self._decode(r)
except requests.RequestException as exc:
logging.warning('attempt %s failed: %s', attempt, exc)
time.sleep(min(2 ** attempt * 2, 60)) # backoff
raise RuntimeError('exhausted retries for ' + url)

@staticmethod
def _decode(resp):
# charset ladder: HTTP header, meta tag, sniff, default UTF-8
if resp.encoding is None or resp.encoding.lower() == 'iso-8859-1':
head = resp.content[:2048].lower()
if b'windows-874' in head or b'tis-620' in head:
resp.encoding = 'windows-874'
else:
resp.encoding = 'utf-8'
return resp.text
```

led_auction_client.py - part 4 of 4: search and parse

```
def load_form(self):
return BeautifulSoup(self._get(FORM_URL), 'html.parser')

def search_by_case(self, case_no, year_be, plaintiff=None,
sale_date=None):
form = {'case_no': case_no, 'year_be': year_be,
'plaintiff': plaintiff or '', 'sale_date': sale_date or ''}
payload = {FIELD_NAMES[k]: v for k, v in form.items()}
self._throttle()
r = self.http.post(RESULT_URL, data=payload,
timeout=self.timeout)
r.raise_for_status()
return self.parse_results(self._decode(r))

@staticmethod
def parse_results(html_text):
# TODO(verify): anchor selectors to the real table headers
soup = BeautifulSoup(html_text, 'html.parser')
return [row.get_text(' | ', strip=True)
for row in soup.find_all('tr')]
```

6.3 JavaScript - Node 18 fetch

Node’s built-in fetch keeps the sample dependency-free. The post-processing comment marks where an HTML parser such as cheerio would slot in once the results structure is captured. Browser-side use of the same code is possible but discouraged: cross-origin policy on asset.led.go.th will not allow it, and mixing public-sector traffic with end-user sessions is exactly what responsible-use guidance says to avoid.

led_auction_fetch.mjs - Node 18+, no dependencies

```
const BASE = 'https://asset.led.go.th';
const FORM_URL = BASE + '/report/reports.asp';
// TODO(verify): capture via DevTools before production use
const RESULT_URL = BASE + '/report/report_result.asp';
const UA = 'Mozilla/5.0 (Windows NT 10.0; Win64; x64)';

const sleep = (ms) => new Promise((r) => setTimeout(r, ms));

export async function searchByCase(params, cookie) {
const res = await fetch(RESULT_URL, {
method: 'POST',
headers: {
'User-Agent': UA,
'Content-Type': 'application/x-www-form-urlencoded',
'Cookie': cookie,
},
body: new URLSearchParams(params),
});
if (!res.ok) throw new Error('HTTP ' + res.status);
const html = await res.text();
// TODO(verify): parse rows with cheerio / DOMParser here
return html;
}
// Politeness: await sleep(2500 + Math.random() * 750) between calls
```

6.4 Politeness pattern

Whatever the language, the same four habits keep an integration welcome: rate-limit to one request every two to three seconds with jitter, schedule bulk work in Thai off-peak hours, cache aggressively (auction results for a closed case never change), and treat any 4xx/5xx burst as a signal to stop rather than to push harder. The retry loop must be bounded - the Python scaffold stops after four attempts with exponential backoff capped at one minute - and a circuit breaker at the scheduler level is worth adding once the integration grows beyond a single worker.

2.5 s	45 s	4	24/7 off
minimum interval between requests	per-request timeout	retry attempts, exponential backoff	schedule bulk jobs in off-peak hours

CHAPTER 7 - A MACHINE-READABLE FACADE FOR A LEGACY FORM

7. OpenAPI Specification Guide

The companion file `openapi/led-auction-openapi.yaml` expresses everything Chapter 4 documented as an OpenAPI 3.0 contract. OpenAPI is used here as a facade: it does not pretend the department offers JSON, but it gives your team a single, lintable, diffable source of truth for paths, parameters, value formats, and the parsed record schema, with the legacy nature of each operation recorded in vendor extensions.

7.1 How the file is organised

The specification declares one server (`https://asset.led.go.th`), three public paths corresponding to E1-E4, and two context paths for the catalogue. Every operation carries `x-verification` with one of two values: verified-page (the form was reached and rendered) or inferred-contract (parameter names pending DevTools capture), plus an x-verify-procedure pointer to Section 3.3. Responses are typed as text/html for the legacy operations and as application/json for the parsed-record operations your own facade will expose once the parser is verified.

led-auction-openapi.yaml - excerpt, facade pattern

```
paths:
/report/reports.asp:
get:
tags: [report-search]
summary: Per-case auction results search form
x-verification: verified-page # reached 2026-10
responses:
'200':
description: HTML form (case no, B.E. year,
plaintiff, sale date)
content:
text/html:
schema: { type: string }
```

7.2 Working with the spec

Import the YAML into Swagger UI or Editor to review the contract visually, or into Postman to generate request scaffolding for the verification session - Postman in particular renders the form-data panel you will be reproducing from DevTools. Keep the spec in the same repository as the client code and let code review reject any commit that changes a parameter name without updating x-verification and the capture date. The record schema (AuctionResultRecord) mirrors Table 5.3 field for field, so the parser, the storage layer, and the documentation converge on one naming. The info.license block deliberately reflects the LED copyright notice rather than implying an open licence; do not relax it without written permission from the department.

CHAPTER 8 - WHAT THAI LAW AND THE SITE NOTICE REQUIRE

8. Legal, Ethics and Responsible Use

This system publishes judicial data about identifiable people and companies, from a government site that asserts ownership of that data. Both facts have consequences for integrators, and this chapter converts them into concrete engineering and governance rules. Nothing here is legal advice; for commercial-scale use, obtain a written opinion and the department's written permission.

8.1 The site's own notice

> *"ข้อมูลต่าง ๆ ในเว็บไซต์นี้ ถือเป็นสมบัติของกรมบังคับคดี ห้ามผู้ใดนำไปใช้ ทำซ้ำ ดัดแปลง แก้ไขข้อมูลดังกล่าวโดยมิได้รับอนุญาต ผู้ใดฝ่าฝืนจะถูกดำเนินคดีตามก์ฎาหมาย"*
>
> Footer declaration observed on asset.led.go.th report pages (verbatim, October 2026)

Translation: "All information on this website is the property of the Legal Execution Department. No person may use, reproduce, or adapt such information without permission; violators will be prosecuted under the law." This is the department's own framing of its rights, displayed on every page of the report family, and any integration plan should start by taking it seriously rather than arguing with it. It interacts with the Thai Copyright Act B.E. 2537 (as amended), under which official state works receive protection with specific exceptions, and with the site's published security and data-governance policies, which include a website policy, a security policy, a data-governance policy, and privacy notices.

8.2 Personal data under the PDPA

The PDPA B.E. 2562 (Personal Data Protection Act) applies in parallel. Auction results identify plaintiffs, defendants, and sometimes bidders - natural persons whose names appear as part of the judicial record. Public accessibility is not the same as reusable consent: a personal-data reuse programme still needs a lawful basis, purpose limitation, and minimisation. The practical translation for this integration is a short list: store case numbers and outcomes rather than party names wherever the use case allows; never redisplay or redistribute party personal data; set a retention period; and document the legal basis in your data inventory. If your product surfaces personal data to end users, route them to the official pages instead of caching the data yourself.

8.3 Responsible-use rules of thumb

Your use case	Recommended route
Look up one case occasionally (personal, investigative)	Use the public web pages directly; no integration needed

Your use case	Recommended route
Research or analytics on aggregate outcomes	Request a bulk extract / permission from LED; try the official open-data catalogue (data.go.th) first
Commercial application or redistribution	Written permission or an MOU with the department before launch
Monitoring new sale results for a watchlist	Low-frequency client (this report's guardrails) plus prominent source attribution and a link back to the official page

Table 8.1 - Route selection by use case.

- Rate: one request every 2-3 seconds with jitter; off-peak bulk windows; aggressive caching of immutable results.
- Identification: a truthful User-Agent with a contact e-mail so the administrator can reach you instead of blocking you.
- Boundaries: never attempt captcha circumvention, credential handling, or security testing of the site - the department publishes a Website Security Standard and treats probing as an incident.
- Accuracy: the official PDF/printed announcements and the court record remain authoritative; a scraped mirror is a convenience copy and should be labelled as such.
- Escalation: if the volume you need is beyond politeness, that is the signal to ask for a proper data channel, not to scale up.

CHAPTER 9 - WHAT CAN GO WRONG AND THE FALLBACK LADDER

9. Risks, Limitations and Mitigations

Every risk below was either observed during this study or follows directly from the platform's architecture. The register is written for an engineering owner: each entry names the failure, its likelihood and impact for a typical integration, and the mitigation that this report's artifacts already implement or recommend.

ID	Risk	L	I	Mitigation
R1	Inferred parameter names differ from the live form	M	H	DevTools verification (3.3); single constants block; OpenAPI x-verification flags
R2	TIS-620 vs UTF-8 mis-decoding corrupts Thai text	M	H	Charset ladder in client; golden tests with known case numbers
R3	Origin slowness / timeouts from some networks (observed)	H	M	45 s timeouts, 4 retries with backoff, off-peak schedule, local cache
R4	Captcha extended to the report family (exists on newbidreg)	M	H	No circumvention; degrade to official pages; request bulk channel

ID	Risk	L	I	Mitigation
R5	Silent schema drift after departmental updates	M	M	Nightly smoke test on a known case; alert on empty or malformed parses
R6	Copyright / PDPA exposure from redistribution	M	H	Chapter 8 rules; minimise personal fields; seek written permission
R7	Data discrepancies vs official announcements	L	M	Label as convenience copy; link to official source; reconciliation job
R8	No SLA, no versioning, no deprecation notice	H	M	Treat as best-effort source; design for graceful degradation

Table 9.1 - Risk register. L = likelihood, I = impact (L/M/H).

9.1 Reading the top risks

R1 is the reason Chapter 3 exists: parameter names in this document were reconstructed from the visible form and classic ASP patterns because the origin throttled programmatic capture, and a wrong name produces an empty result set rather than an error, which makes the failure silent. R3 was observed directly - repeated connection timeouts from a datacentre address while normal browsing worked - and it argues for scheduling and caching more than for faster retries. R4 deserves respect: the sibling catalogue already prompts for a confirmation code, so the day the report family adds one, the correct response is to stop and switch channels, not to work around it. R6 is the risk that outlives the code: data you obtained politely can still become a legal problem the moment it is redistributed without permission, which is why Chapter 8 precedes any scaling conversation.

CHAPTER 10 - REFERENCE LISTS AND CHECKLIST

10. Appendix

The appendix collects the reference material that supports the main chapters: a sample of the enforcement-office enumeration, verified case-number examples, the go-live checklist, the reference URLs used throughout this report, and the bundle's file map.

A. Enforcement office list (sample)

The dropdown on E3/E4 enumerates the department's offices and branches; the sample below shows the head of the observed list to illustrate structure and naming. The full list is embedded as a YAML enum in the OpenAPI file. Office count exceeds one hundred entries, including Bangkok divisions and provincial branches.

Thai label (dropdown entry)	Suggested code
กระบี่ (Krabi)	`KRABI`

Thai label (dropdown entry)	Suggested code
กรุงเทพ กองล้มละลาง 1	BKK-BANK-1
กรุงเทพ กองล้มละลาง 2	BKK-BANK-2
กรุงเทพ สำนักฟื้นฟูกิจการของลูกหนี้	BKK-REHAB
กาญจนบุรี (Kanchanaburi)	KANB
ขอนแก่น (Khon Kaen)	KHON
เชียงใหม่ (Chiang Mai)	CHMA
นครราชสีมา (Nakhon Ratchasima)	NAKO
นนทบุรี (Nonthaburi)	NONB
สมุทรปราการ (Samut Prakan)	SAMP

Table 10.1 - Office enumeration sample (10 of 100+ entries).

B. Verified case-number examples

Red case number	Seen in	Reading
ย.1151/2545	Asset-detail page referenced by an asset-management company	Prefix ย., file 1151, B.E. 2545 (C.E. 2002)
ผป.1730/2562	Provincial LED office auction announcement	Prefix ผป., file 1730, B.E. 2562 (C.E. 2019)

Table 10.2 - Red case numbers verified in public announcements.

C. Go-live checklist

- DevTools verification completed; parameter names and submission endpoint recorded with a capture date in the OpenAPI x-verification fields.
- Parser anchored to the real results-table headers; golden-sample test stored for one known case and one known office-date window.
- Charset ladder confirmed against both UTF-8 and (if ever served) TIS-620 responses; Thai names render correctly end to end.
- Rate limiter, timeouts, retry-with-backoff, and circuit breaker enabled; bulk jobs scheduled off-peak.
- Storage schema keeps party personal data out unless a lawful basis is documented; retention period set.

- Source attribution and link-back to the official page implemented in every surface that displays the data.
- Contact established with LED (itsupport@led.mail.go.th) and the permission question answered for your use case.

D. Reference URLs

Resource	URL
Per-case results search (E1)	`https://asset.led.go.th/report/reports.asp`
Office-and-date results browser (E3)	`https://asset.led.go.th/report/report.asp`
Newer office-and-date browser (E4)	`https://asset.led.go.th/report_new/report.asp`
Auction asset catalogue (E5)	`https://asset.led.go.th/newbidreg/`
Asset detail pattern (E6)	`https://asset.led.go.th/newbidreg/asset_open.asp`
Departmental portal	`https://www.led.go.th/`
e-Service catalogue	`https://www.led.go.th/th/eservice`
Official open-data catalogue	`https://data.go.th/`

Table 10.3 - Reference URLs (all verified reachable via normal browsing, October 2026).

E. Bundle file map

Path in bundle	Contents
`LED_Auction_Results_API_Report.pdf`	This report
`openapi/led-auction-openapi.yaml`	OpenAPI 3.0 facade: paths E1-E6, parameters, record schema, office enum
`samples/led_auction_client.py`	Python client: session, throttle, retries, charset ladder, parse stub
`samples/led_auction_curl.sh`	cURL verification recipe from Chapter 6.1
`samples/led_auction_fetch.mjs`	Node 18+ fetch sample from Chapter 6.3
`assets/cover.html`	Editable HTML source of the report cover

Table 10.4 - Documentation bundle contents.